

```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
file_path = 'Electric_Vehicle_Population_Data.csv' # Replace with your file path
data = pd.read_csv(file_path)

# Display basic dataset information
print("Dataset Information:")
print(data.info())
print("\nMissing Values:")
print(data.isnull().sum())

# Summary statistics
print("\nSummary Statistics:")
print(data.describe())
```

```
3 State 50000 non-null object
4 Postal Code 50000 non-null int64
5 Model Year 50000 non-null int64
6 Make 50000 non-null object
7 Model 50000 non-null object
8 Electric Vehicle Type 50000 non-null object
9 Clean Alternative Fuel Vehicle (CAFV) Eligibility 50000 non-null object
10 Electric Range 50000 non-null int64
11 Base MSRP 50000 non-null int64
12 Legislative District 50000 non-null int64
13 DOL Vehicle ID 50000 non-null int64
14 Vehicle Location 50000 non-null object
15 Electric Utility 50000 non-null object
16 2020 Census Tract 50000 non-null int64
dtypes: int64(7), object(10)
memory usage: 6.5+ MB
None
```

```
Missing Values:
VIN (1-10) 0
County 0
City 0
State 0
Postal Code 0
Model Year 0
Make 0
Model 0
Electric Vehicle Type 0
Clean Alternative Fuel Vehicle (CAFV) Eligibility 0
Electric Range 0
Base MSRP 0
Legislative District 0
DOL Vehicle ID 0
Vehicle Location 0
Electric Utility 0
2020 Census Tract 0
dtype: int64
```

```
Summary Statistics:
Postal Code Model Year Electric Range Base MSRP \
count 50000.000000 50000.000000 50000.000000 50000.000000
mean 98176.078600 2020.614720 59.194240 1073.009300
std 209.316572 2.931493 92.316356 8208.839278
min 98001.000000 1998.000000 0.000000 0.000000
25% 98034.000000 2019.000000 0.000000 0.000000
50% 98109.000000 2022.000000 0.000000 0.000000
75% 98188.000000 2023.000000 82.000000 0.000000
max 99362.000000 2024.000000 337.000000 184400.000000
```

```
Legislative District DOL Vehicle ID 2020 Census Tract
count 50000.000000 5.000000e+04 5.000000e+04
mean 31.956420 2.208389e+08 5.303355e+10
std 14.513591 7.590555e+07 1.177161e+07
min 1.000000 2.770200e+04 5.300796e+10
25% 19.000000 1.830719e+08 5.303301e+10
50% 36.000000 2.286586e+08 5.303302e+10
75% 45.000000 2.546139e+08 5.303303e+10
max 49.000000 4.789259e+08 5.307794e+10
```

```
# Checking unique values in categorical columns
categorical_columns = ['County', 'City', 'State', 'Make', 'Model', 'Electric Vehicle Type']
for col in categorical_columns:
    print(f"\nUnique values in {col}: {data[col].nunique()}")
```



```
Unique values in County: 20
```

```
Unique values in City: 152

Unique values in State: 1

Unique values in Make: 38

Unique values in Model: 132

Unique values in Electric Vehicle Type: 2


# Distribution of Electric Range
plt.figure(figsize=(8, 6))
sns.histplot(data['Electric Range'], kde=True, bins=30, color='blue', alpha=0.7)
plt.title('Distribution of Electric Range')
plt.xlabel('Electric Range (miles)')
plt.ylabel('Frequency')
plt.show()


# Count plot for Electric Vehicle Type
plt.figure(figsize=(8, 6))
sns.countplot(y='Electric Vehicle Type', data=data, palette='muted', order=data['Electric Vehicle Type'].value_counts().index)
plt.title('Count of Electric Vehicle Types')
plt.xlabel('Count')
plt.ylabel('Vehicle Type')
plt.show()

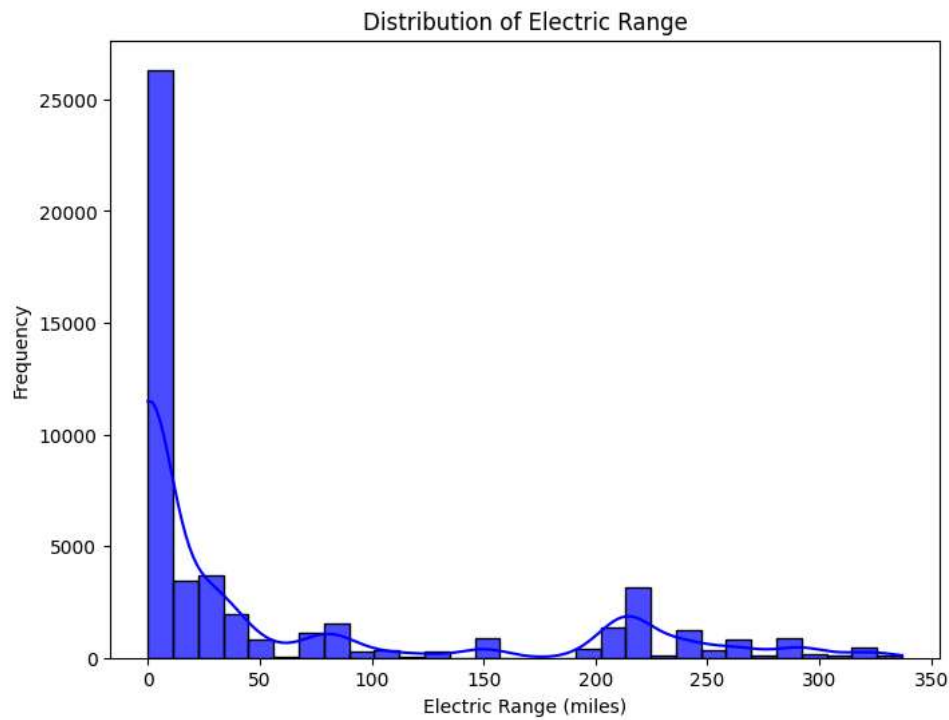

# Top 10 Vehicle Makes
top_makes = data['Make'].value_counts().head(10)
plt.figure(figsize=(10, 6))
sns.barplot(x=top_makes.values, y=top_makes.index, palette='coolwarm')
plt.title('Top 10 Vehicle Makes')
plt.xlabel('Count')
plt.ylabel('Make')
plt.show()


# Scatterplot matrix for numeric columns
numeric_columns = ['Model Year', 'Electric Range', 'Base MSRP', '2020 Census Tract']
sampled_data = data[numeric_columns].sample(2000, random_state=42) # Sampling for efficiency
sns.pairplot(sampled_data, diag_kind='kde', corner=True, plot_kws={'alpha': 0.5})
plt.show()


# Top models in the dataset
top_models = data['Model'].value_counts().head(10)
plt.figure(figsize=(10, 6))
sns.barplot(x=top_models.values, y=top_models.index, palette='coolwarm')
plt.title('Top 10 Electric Vehicle Models')
plt.xlabel('Count')
plt.ylabel('Model')
plt.show()


# Distribution of vehicle makes across top counties
top_counties = data['County'].value_counts().head(10).index
plt.figure(figsize=(10, 6))
sns.countplot(data=data[data['County'].isin(top_counties)], y='County', palette='viridis', order=top_counties)
plt.title('Top 10 Counties with Most Electric Vehicles')
plt.xlabel('Count')
plt.ylabel('County')
plt.show()

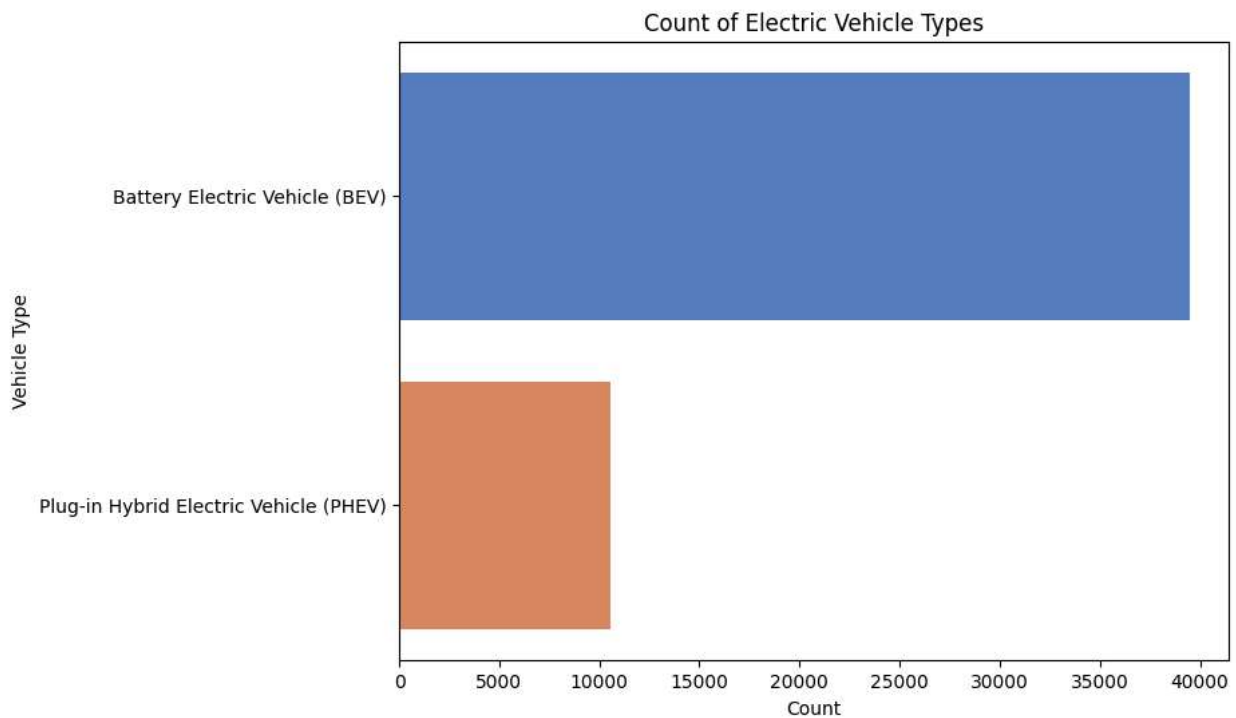

# Electric Range vs Base MSRP
plt.figure(figsize=(10, 6))
sns.scatterplot(data=data, x='Electric Range', y='Base MSRP', hue='Electric Vehicle Type', alpha=0.7)
plt.title('Electric Range vs Base MSRP by Vehicle Type')
plt.xlabel('Electric Range (miles)')
plt.ylabel('Base MSRP ($)')
plt.legend(title='Vehicle Type')
plt.show()
```



```
<ipython-input-10-c5ad55015fb1>:11: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and specify each category with its corresponding color.

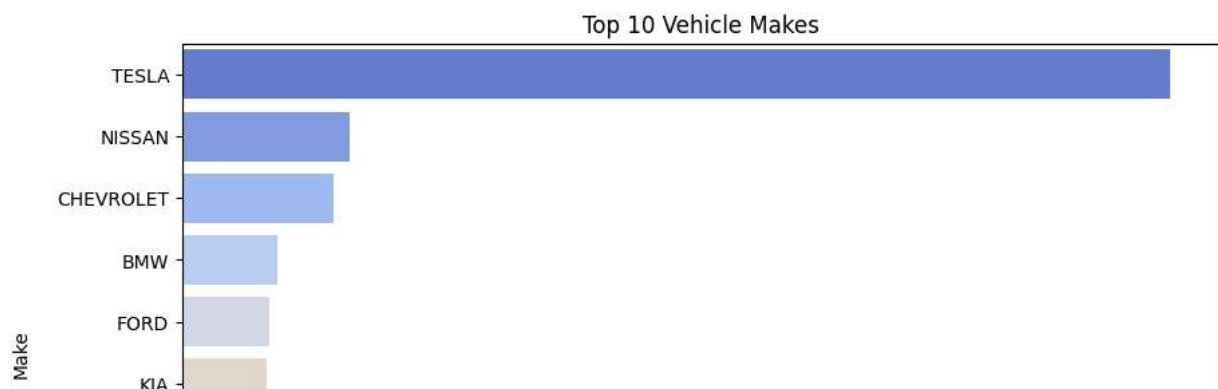
```
sns.countplot(y='Electric Vehicle Type', data=data, palette='muted', order=data['Electric Vehicle Type'].value_counts().index)
```

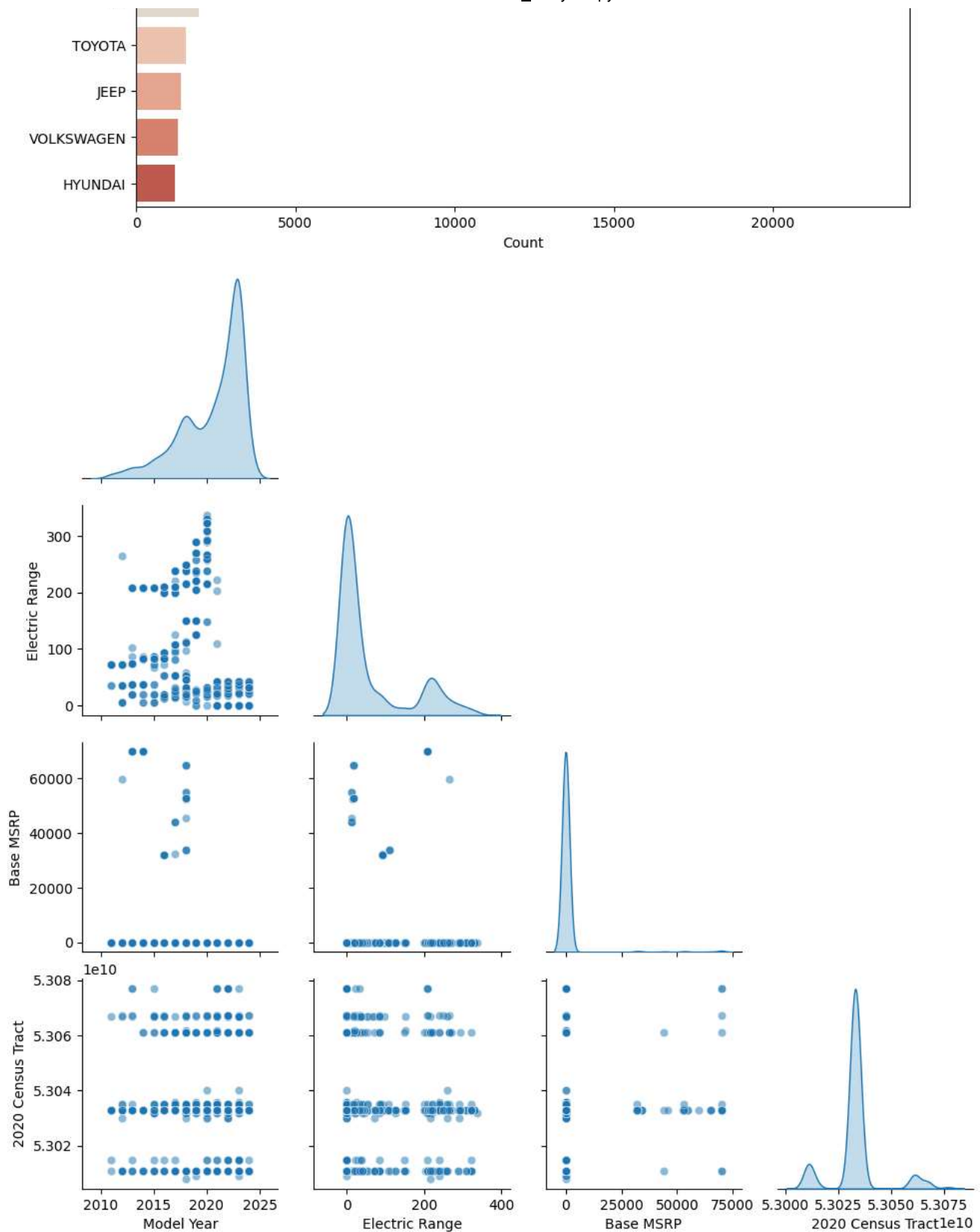


```
<ipython-input-10-c5ad55015fb1>:20: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and specify each category with its corresponding color.

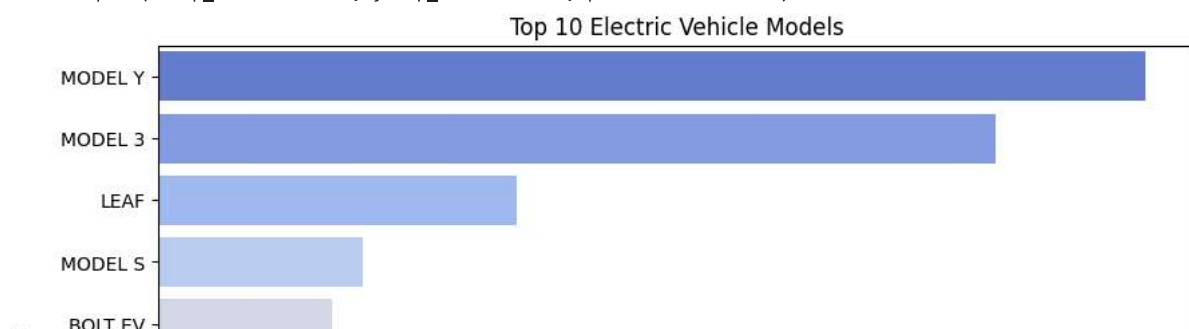
```
sns.barplot(x=top_makes.values, y=top_makes.index, palette='coolwarm')
```

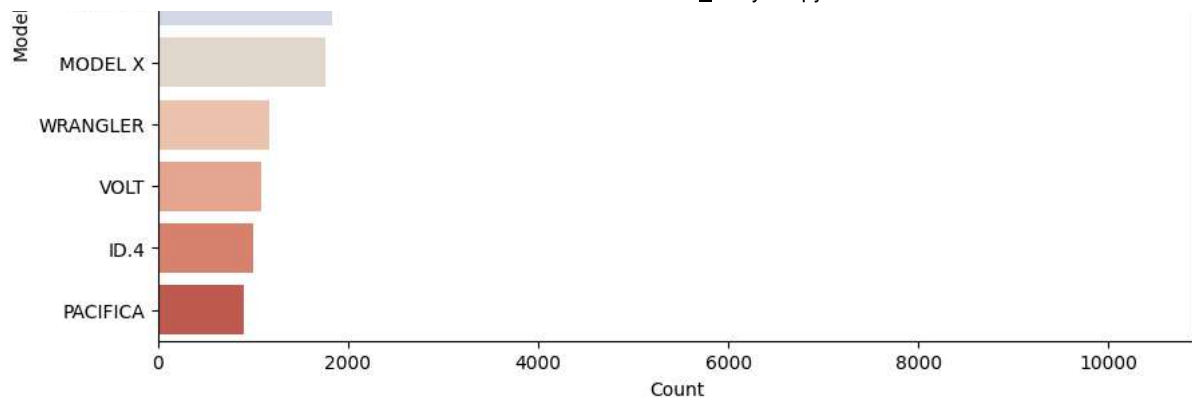




<ipython-input-10-c5ad55015fb1>:35: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and
 sns.barplot(x=top_models.values, y=top_models.index, palette='coolwarm')

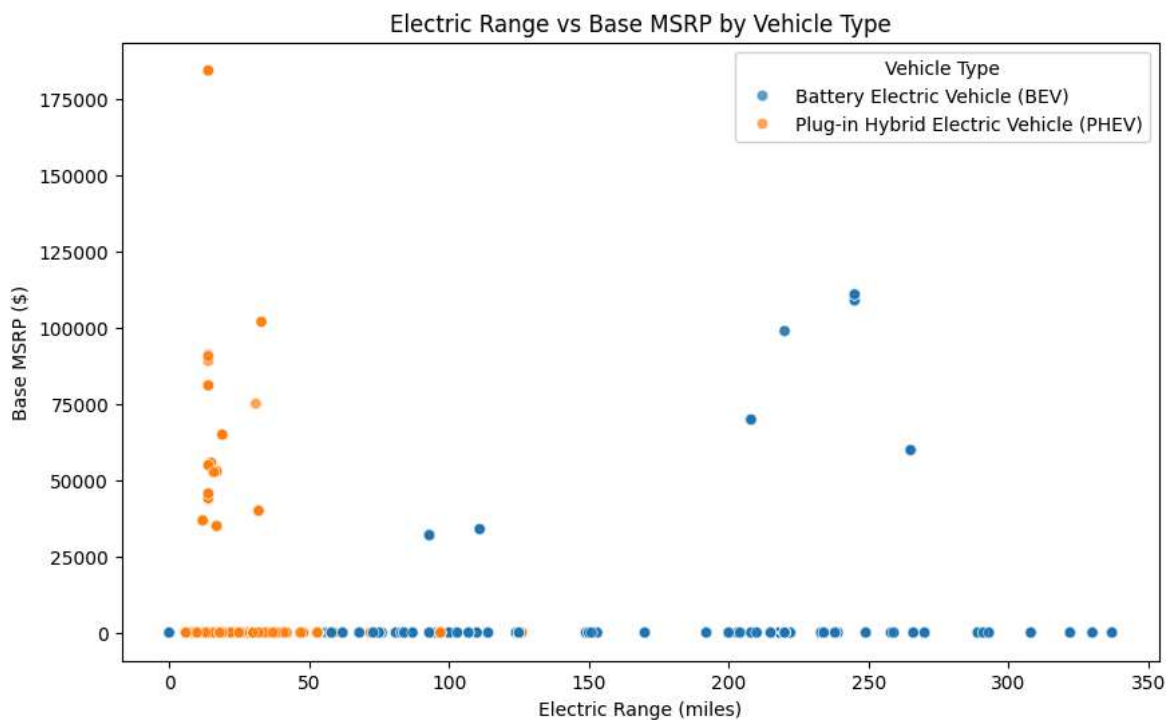
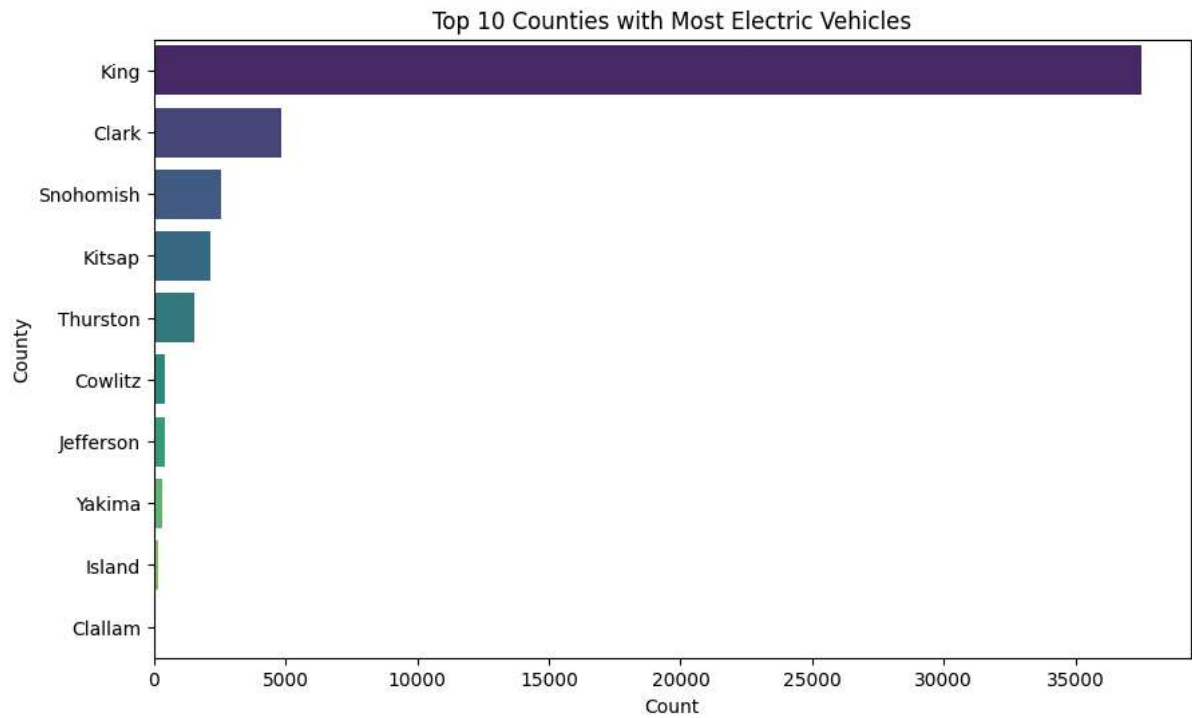




```
<ipython-input-10-c5ad55015fb1>:44: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set

```
sns.countplot(data=data[data['County'].isin(top_counties)], y='County', palette='viridis', order=top_counties)
```



```
➡ Location heatmap saved as 'location_heatmap.html'.
```

